

Documentazione

Valyrix

Valyrix è un linguaggio di programmazione general-purpose.

Indice

1. [Introduzione](#)
2. [Guida Rapida](#)
3. [Sintassi](#)
4. [Semantica](#)
5. [Implementazione](#)
6. [Programmi di Test](#)

1. Introduzione

Valyrix è un linguaggio di programmazione imperativo tipizzato general-purpose ma non presenta la possibilità di istanziare classi.

Caratteristiche principali

- **Tipizzazione statica** con i tipi di dato più comuni (int, dec, bool, string, char)
- **Due sezioni distinte**: una sezione per dichiarare le variabili, ed una sezione per il codice eseguibile.
- **Espressioni aritmetico-logiche** complete con precedenza degli operatori.
- **Costrutti iterativi e condizionali**: `while`, `for`, `if - else`.
- **Arrays** come strutture dati statiche monodimensionali.
- **Zucchero sintattico**: assegnamenti composti (`+=`, `-=`, `*=`, `/=`), incremento e decremento unitario (`++`, `--`), operatore ternario (`_ ? _ : _`), e interpolazione di espressioni nelle stringhe (`${expr}`).
- **Gestione degli errori**: divisione per zero, accesso fuori dai limiti di un array, variabile non dichiarata, errori di tipo (tutti gestiti con messaggi esplicativi).
- **Operatore Non Deterministico** presenza di un operatore che esegue una istruzione casuale tra un insieme di comandi
- **Casting esplicito** casting esplicito, permette conversione tra tutti i tipi di dato semplici e tra tutti i tipi di dato array
- **Flusso di Controllo Condizionato** Supporto per un costrutto iterativo con possibilità di uscita prematura dall'iterazione tramite un apposito comando. Compatibile con solo con il ciclo `for`.

Contesto applicativo

Valyrix punta ad essere un linguaggio semplice con una sintassi simile a C e Java.

2. Guida Rapida

Requisiti

- **Java 11** o superiore, necessario per ANTLR
- **ANTLR 4** (versione 4.11 o superiore)
- **MAVEN** (versione 3.7.1 o superiore)

Installazione programma

```
mvn clean install
mvn clean compile
```

`mvn clean compile` pulisce i file generati in precedenza e compila il progetto. `mvn clean install`, oltre a pulire e compilare, esegue i test, crea il pacchetto del progetto e lo installa nel repository locale di Maven.

Eseguire un programma

```
mvn exec:java "-Dexec.mainClass=progetto.Main" "-Dexec.args=.\programs\helloWorld.vlrx"
```

Questo esegue l'interprete del linguaggio sul programma `helloWorld.vlrx` contenuto nella cartella `programs`.

Per effettuare eventuali test :

```
mvn test-compile
mvn test-compile exec:java "-Dexec.mainClass=Test" "-Dexec.classpathScope=test"
```

Hello World

Un programma per iniziare con Valyrix .

```
string name;
input name;
print "Ciao " :: name
```

Input

Mario

Output

Ciao Mario

Un esempio di programma in Valyrix che effettua semplici operazioni aritmiche

```
int num = 5;
int i;
int end = 10;
print "tabellina del 5";
for i from 0 to end{
```

```
print num :: " X " :: ${i + 1} :: " = " :: ${i + 1} * num}
}
```

Output

```
tabellina del 5
5 X 1 = 5
5 X 2 = 10
5 X 3 = 15
5 X 4 = 20
5 X 5 = 25
5 X 6 = 30
5 X 7 = 35
5 X 8 = 40
5 X 9 = 45
5 X 10 = 50
```

3. Sintassi

3.1 Struttura di un programma

Un programma Valyrix è composto da due sezioni:

```
main : decl com EOF
```

- **decl** : sezione opzionale dichiarativa per dichiarare e inizializzare le variabili che verranno usate nel programma.
- **com** : sezione eseguibile. Contiene il corpo del programma rappresentato come serie di comandi terminati da ";" , l'ultima istruzione di ogni blocco (anche nei blocchi annidati come nell'esempio precedente) non necessita del terminatore.

3.2 Tipi di dato

Tipo	Descrizione	Esempio di valore
Int	Intero con segno	42 , -3
dec	Numero in virgola mobile	3.14 , -0.5
Bool	Valore booleano	true , false
String	Stringa di testo	"ciao"
char	Carattere ascii	'a' , 'b'

3.3 Dichiarazione di variabili

```
int x = 8;
dec z;    // la variabile deve essere valorizzata prima dell'utilizzo
```

```
string s = "prova";
int[] array = [1, 2, 3];
```

Nella fase di dichiarazione l'inizializzazione è opzionale, ma deve essere effettuata nel codice prima dell'utilizzo della variabile, in caso contrario avviene un errore di mancata inizializzazione. Le variabili sono globali e posso essere dichiarate solo all'inizio del programma, non esiste di conseguenza shadowing delle variabili nel codice.

3.4 Assegnamenti

```
int x = 30;
int y = 5;
x += 5;    // equivalente a x = x + 5
x -= 2;    // equivalente a x = x - 2
x *= 3;    // equivalente a x = x * 3
x /= 4;    // equivalente a x = x / 4
++x;       // pre-incremento, incremento unitario prima della valutazione
x++;       // post-incremento, incremento unitario dopo la valutazione
--x;       // pre-decremento, decremento unitario prima della valutazione
x--;       // post-decremento, decremento unitario dopo la valutazione
y = x++ - 5; // y = 30 - 5 --> y = 25, x = 31 poiche' l'incremento viene effettuato dopo aver terminato l'istruzione
y = x++ - 5; // y = 32 - 5 --> y = 27, x = 32 poiche' in questo caso l'incremento viene effettuato subito
```

Gli operatori di pre e post incremento / decremento possono essere usati sia da soli che all'interno delle espressioni.

3.5 Espressioni

Le espressioni supportano la **precedenza standard** degli operatori (dal più basso al più alto):

Livello	Operatori
1	_ ? _ : _ (ternario)
2	\ \ , && (or, and logici)
3	== , != , < , > , <= , >= (comparatori)
4	+ , - (somma, sottrazione)
5	* , / , % (moltiplicazione, divisione, modulo)
6	! (negazione logica)
7	Accesso array arr[i]

Operatore ternario:

```
int numero = 5;
String msg = (numero % 2 == 0) ? "pari" : "dispari"
```

La variabile `msg` viene assegnata con la stringa *"pari"* se la variabile `numero` è divisibile per 2 con resto 0, altrimenti viene assegnato con la stringa *"dispari"*.

3.6 Costrutti di controllo

Condizionale:

```
if (numero < 0) {  
    print "Numero negativo"  
} else {  
    print "Numero positivo"  
}
```

Ciclo while:

```
Int i = 0;  
while (i < 5) {  
    print "numero " :: ${i};  
    i++  
}
```

Ciclo for:

```
for i from 1 to 10 {  
    print "numero " :: ${i};  
    if(i == 5){  
        break  
    }  
}
```

Il ciclo `for i from a to b` itera `i` da `a` incluso a `b` escluso `a` e `b` possono essere solo numeri interi. I cicli `while` e `for` possono terminare anticipatamente con l'utilizzo dell'istruzione `"break"`.

3.7 Stampa e interpolazione nelle stringhe

```
print "Testo semplice";  
print i"Risultato: ${x + y} unità";  
print i"Il doppio di ${n} è ${n * 2}";
```

Le stringhe interpolate sono comprese tra `"${"` e `"}"`, e contengono espressioni di qualsiasi tipo e possono essere concatenate con le stringhe semplici

3.8 Array

```
int[] a = [1, 2, 3];  
int n = 2;  
print a;
```

```
a[1] = 5 * n;  
print a
```

3.9 Uscita dal programma

```
exit;    // termina immediatamente l'esecuzione del programma
```

3.10 Commenti

```
// Commento su una riga  
/* Commento  
   su più righe */
```

3.11 Regole lessicali principali

- Gli identificatori iniziano con una lettera , seguiti da lettere, cifre.
- Le keyword sono case-sensitive e riservate (ad esempio, `if` , `while` , `for` , `true` , `false` , i nomi dei tipi).
- I blocchi sono delimitati da `{` e `}` .
- Ogni statement termina con `;` ; tranne l'ultimo di ogni blocco.
- Gli spazi bianchi e le tabulazioni sono ignorati.

4. Semantica

4.1 Tipizzazione

Valyrix adotta **tipizzazione statica**: ogni variabile deve essere dichiarata con un tipo esplicito, e il tipo non cambia durante l'esecuzione. Le verifiche di tipo avvengono prima di eseguire un programma tramite un **type system**, il quale riporta errori segnalati con messaggi descrittivi. Gli errori gestiti staticamente includono:

Errore	Messaggio
Variabile non dichiarata	<i>Variable a assigned but never declared. @1:0</i>
Tipo incompatibile	Type mismatch: numeric expression expected. @3:10
Assegnamento di tipo errato	<i>assigned value [1,2] of type ArrayType.INT is not compatible with type SimpleType.INT @1:0</i>

Warning	Messaggio
Casting critico	Warning: unsafe cast from char to int @20:4

questo warning appare quando si fanno dei casting tra valori solitamente non compatibili es: `char --> int`, `bool --> char`, `int --> bool`

4.2 Visibilità e scoping

Le variabili in Valyrix sono definite solo all'inizio del programma

4.3 Valutazione delle espressioni

Le espressioni sono valutate in modo standard: tutti gli operandi vengono valutati prima dell'applicazione dell'operatore. Gli operatori logici `&&` e `||` **non** adottano la semantica short-circuit.

4.4 Gerarchia dei tipi

```
Bool
String
Char
Dec
Int
```

Il linguaggio nella versione attuale prevede **conversioni esplicite** utilizzando la sintassi del tipo: **(TIPO) VAR**.

NB :

Il tipo `dec` è implementato come tipo `double` in java .

4.5 Gestione degli errori a runtime

Gli errori a tempo d'esecuzione vengono intercettati dall'interprete e riportati con messaggi esplicativi, **senza terminare** bruscamente il programma. Gli errori gestiti includono:

Errore	Messaggio
Divisione per zero	<i>"Division by zero"</i>
Indice array fuori limite	<i>"Index out of bounds 7 array size 2"</i>
Esponente negativo con base 0	<i>"Exponential base cannot be zero with exponent lower than 0"</i>

4.6 Semantica operativa

Semantica dell'if-else

Per il comando:

$$if(e) \ c_1 \ else \ c_2$$

si hanno due regole di transizione, a seconda del valore della condizione (true o false).

La memoria σ non viene modificata durante la scelta del ramo.

Se la condizione e valutata nella memoria σ restituisce `true` , viene eseguito il comando c_1 altrimenti se la valutazione restituisce `false` viene eseguito il comando c_2

Condizione vera

$$\frac{e \rightarrow true}{(\sigma, if(e)\{c_1\} else \{c_2\}) \rightarrow (\sigma, c_1)}$$

Condizione falsa

$$\frac{e \rightarrow false}{(\sigma, if(e)\{c_1\} else \{c_2\}) \rightarrow (\sigma, c_2)}$$

Ciclo while

$$\text{While} \frac{-}{(\bar{\sigma}, while(e)\{c\}) \rightarrow (\bar{\sigma}, if(e)\{c; while(e)\{c\}\})}$$

Sequenza

$$\text{SeqP} \frac{(\bar{\sigma}, c_1) \rightarrow (\bar{\sigma}', c'_1)}{(\bar{\sigma}, c_1; c_2) \rightarrow (\bar{\sigma}', c'_1; c_2)} \quad \text{SeqE} \frac{-}{(\bar{\sigma}, \epsilon; c) \rightarrow (\bar{\sigma}, c)}$$

5. Implementazione

5.1 Struttura del progetto

```
documentazione.md
documentazione.pdf
pom.xml
programs/  programmi di test
sources/
├─ main
│   └─ java/
│       ├── config/
│       │   ├── ProgramConfig.java
│       │   └─ config.json - ProgramConfig.java
│       └─ progetto/
│           ├── exception/ tutti i file exception
│           ├── type/ tutti i file con le tipologie
│           ├── utils/ file contenenti metodi utilizzati nel progetto
│           ├── value/ tutti i file value
│           ├── Interprete.java
│           ├── Main.java
│           ├── Mem.java
│           └─ TypedImoTS.java
│   └─ antlr4/
│       └─ progetto/Linguaggio.g4
│   └─ resources /
└─ test/
    └─ java/
        ├── TestClasses/ tutte le tipologie di test divise per classi
        ├── TestMenu/ tutti i file menu compreso il main che lo gestisce
        └─ Test.java
```

5.2 L'interprete

L'interprete è implementato come un **visitor** generato da ANTLR4 (`LinguaggioVisitor`) in Java. La classe principale `Interprete` estende `LinguaggioVisitor` e sovrascrive il metodo `visit*` per ogni produzione

della grammatica.

Il metodo `visitMain` è il punto iniziale del programma e poi avvia la visita della sezione `decl`.

5.3 Zucchero sintattico

Gli operatori composti e gli operatori di incremento/decremento unitario sono gestiti direttamente nel visitor.

Come metodo alternativo serve per la scrittura delle espressioni che verranno interpretate come stringhe `${expr}`.

Nel metodo `print` non serve utilizzare questo costrutto poiché viene effettuato il casting a stringa in modo implicito.

5.4 Non determinismo

L'operatore non deterministico esegue un comando tratto da una lista con una probabilità proporzionale alla dimensione della lista. Ad esempio, la valutazione del comando `x = 1 @ x = 2` esegue il comando `x = 1` con probabilità 1/2 (o il comando `x = 2` con probabilità 1/2), mentre la valutazione del comando `x = 1 @ x = 2 @ x = 3` esegue il comando `x = 1` con probabilità 1/3.

Per ottenere input da tastiera si utilizza il comando `input ( var )` dove `var` è una variabile di tipo stringa in cui verrà salvata una serie di caratteri fino allo `\n` prelevati da `console`.

5.5 Flusso di controllo condizionato

All'interno del linguaggio esiste un costrutto (`break`) per interrompere le iterazioni di un ciclo e causarne l'uscita prematura. Il costrutto `for` prevede inoltre un blocco di codice da eseguire se e solo se la condizione d'iterazione diventa falsa (quindi il comando di uscita prematura non viene eseguito).

Ad esempio, nel seguente comando `for i from 2 to (n-1) { if (n % i == 0) break } { print "Not prime!" }` la stampa a video avviene solo quando `n` non è un numero primo.

Viceversa, se `n` è un numero primo nulla viene stampato ed il ciclo viene interrotto, ad un certo punto, dal comando `break`.

Il linguaggio supporta un costrutto condizionale a scelta multipla (`switch`), nel quale l'espressione di controllo può assumere uno dei valori tra quelli a scelta. Solo il blocco di codice relativo ad una delle scelte viene eseguito.

Il costrutto può avere o meno una scelta predefinita (`default`) da eseguire quando nessuna scelta è percorribile.

5.6 Difficoltà tecniche

- **Interpolazione nelle stringhe:** è stato necessario gestire con attenzione i simboli `{` e `}`, usati sia per i blocchi di codice sia per le espressioni all'interno delle stringhe.
- **Tipizzazione statica:** ANTLR non permette di definire direttamente le regole di tipaggio. Per questo, i tipi vengono controllati durante una prima visita del programma tramite un apposito interprete (`type system`), mentre l'esecuzione avviene durante una seconda visita tramite l'interprete.
- **Gestione file progetto:** abbiamo riscontrato delle difficoltà a far interagire correttamente `MAVEN`, `ANTLR` ed i file sorgenti.

6. Programmi di Test

helloworld.vlrx – Hello world

```
int numeroFortunato = 5;
print "Hello World!" :: "\nnumero di oggi " :: numeroFortunato
```

Output atteso

Hello World!
numero di oggi 5

estrazione.vlrx – estrazione casuale

```
int scelta = 0;
int ind;
string t;
int estratto;
for ind from 0 to 3{
    print "tenta la fortuna inserendo un numero intero";
    input t;
    scelta = (int) t;
    <<estratto = scelta @ estratto = scelta @ estratto = -100>>;
    if(estratto != -100){
        print "hai vinto "
    }
    else{
        if(scelta == -100){
            print "non avevi possibilita' di vincere, hai messo il numero perdente"
        }
        else{
            print "hai perso "
        }
    }
}
```

Output atteso

tenta la fortuna inserendo un numero intero
5
hai vinto

tenta la fortuna inserendo un numero intero
3
hai vinto

tenta la fortuna inserendo un numero intero

mod_array_int.vlrx – Modifica di un array intero inserito da console

```

int[] x = [0, 0, 0, 0, 0];
int num;
int ind;
string t;
print "Quanti numeri vuoi inserire ?";
input t;
num = (int) t;
if(num > 5){
    print("Mi dispiace sono troppi non ne voglio di piu' di 5 ;) ");
    num = 5
};
if(num < 1){
    print("Se non vuoi inserire numeri potevi anche non avviare il programma ;) ");
    exit
};
for ind from 0 to num {
    print "inserire un numero intero:";
    input t;
    x[ind] = (int) t
};
print "Array inserito " :: x;
for ind from 0 to num {
    if(ind % 2 == 0){
        x[ind] = x[ind] * 5
    }
    else{
        x[ind] = x[ind] - 2
    }
};
print "Array modificato " :: x

```

Output atteso

Quanti numeri vuoi inserire ?

4

inserire un numero intero:

1

inserire un numero intero:

2

inserire un numero intero:

3

inserire un numero intero:

4

Array inserito [1, 2, 3, 4, 0]

Array modificato [5, 0, 15, 2, 0]

mini_menu_scelta.vlrx – Mini menu a scelta

```
int a;
int scelta;
int i;
int x;
string s;
print "Benvenuto";
print "Scegli quale tra le opzioni vuoi provare";
print "1 - countdown da 10 ";
print "2 - while";
input s;
a = 0;
scelta = (int) s;
print "";
if (scelta == 1 ) {
    for i from 0 to 10 {
        // print "for";
        a = i;
        print (string) a
    }
};
if(scelta == 2 ) {
    x = 0;
    while(x < 5){
        if(x == 2){
            print ">>fine anticipata";
            break
        }
        else{
            print ">>ciao"
        };
        x++;
    }
}
else{
    print "Scelta non valida"
}
```

Output atteso

Benvenuto

Scegli quale tra le opzioni vuoi provare

1 - countdown da 10

2 - while

1

0
1
2
3
4
5
6
7
8
9

Benvenuto

Scegli quale tra le opzioni vuoi provare

1 - countdown da 10

2 - while

2

ciao

ciao

fine anticipata